UNIVERSIDAD POLITÉCNICA BICENTENARIO

# Práctica 1:  Control de dispositivos básicos


Presenta:

**Jesús Ángel Bonilla Duran**

**Viridiana Santibáñez Carrillo**

**Jocelyn Rodriguez Torres**

Matricula:

**24033407**

**24033066**

**24033436**


Asignatura:

**Sistemas Embebidos**

Profesor:

**Hugo Antonio Méndez Guzmán**

Silao 15 de octubre de 2026

Jesús Ángel Bonilla Duran Viridiana Santibañez Carrillo Jocelyn Rodriguez Torres

# Contenido

**Introducción** ..... 3
**Marco Teórico** ..... 3
Sistemas programables ..... 3
Microcontrolador ESP32 ..... 3
Programación del ESP32 ..... 4
Simulación ..... 4
Push buttons como entradas digitales ..... 5
**Desarrollo** ..... 5
Material requerido ..... 5
Descripción del código ..... 8
**Resultados** ..... 8
**Observaciones y Conclusiones** ..... 9
**Referencias** ..... 9

# Lista de Figuras

Figura 1 Circuito de control de un LED mediante dos botones pulsadores utilizando el ESP32. ..... 6
Figura 2. Circuito simulado en Wokwi para el control de un motor mediante el ESP32 y un módulo de relevador. ..... 6
Figura 3. Circuito de control de un LED mediante un módulo de relevador y dos botones pulsadores utilizando el ESP32 ..... 7
Figura 4.Montaje físico cuando el foco está apagado ..... 8
Figura 5. Foco encendido ..... 9

**PRACTICA 1_ SISTEMAS EMEBIDOS**

**Jesús Ángel Bonilla Duran** **Viridiana Santibañez Carrillo** **Jocelyn Rodriguez Torres**

# Introducción

Las entradas y salidas digitales son elementos fundamentales en los sistemas de control, ya que permiten recibir señales y ejecutar acciones mediante un microcontrolador, estas funciones se utilizan en aplicaciones de automatización para controlar dispositivos electrónicos a partir de las instrucciones establecidas en un programa.

En esta práctica se utiliza una tarjeta ESP32 para controlar un LED mediante dos botones pulsadores, el primer botón permite encender el LED, mientras que el segundo permite apagarlo para detectar las pulsaciones se utiliza la configuración INPUT_PULLUP, que habilita las resistencias internas de las entradas digitales.

Además, se emplea la comunicación serial para mostrar mensajes que indican cuándo se presiona cada botón. De esta manera, se busca comprender la configuración de los pines GPIO, el control de una salida digital y la lectura de entradas mediante programación.

# Marco Teórico

## Sistemas programables

Un sistema programable es un sistema electrónico o computacional capaz de realizar diferentes funciones mediante un conjunto de instrucciones almacenadas en un programa a diferencia de un sistema de función fija, puede modificar su comportamiento al cambiar el programa sin necesidad de alterar físicamente sus componentes estos sistemas se utilizan en áreas como la robótica, automatización, control y dispositivos electrónicos.

## Microcontrolador ESP32

El ESP32 es un microcontrolador y sistema en chip (SoC) desarrollado por Espressif Systems, que integra conectividad Wi-Fi y Bluetooth un SoC es un circuito integrado que reúne en una sola plataforma un sistema electrónico completo para una aplicación determinada, debido a sus características, el ESP32 puede utilizarse en diferentes tipos de dispositivos y proyectos relacionados con sistemas embebidos e Internet de las Cosas (IoT) (Nick, s.f.).

El ESP32 cuenta con diferentes periféricos que permiten la conexión y control de otros componentes electrónicos entre ellos se encuentran las entradas y salidas digitales, mediante las cuales es posible recibir señales de dispositivos externos y controlar diferentes elementos conectados al microcontrolador estas características permiten utilizarlo para aplicaciones de control, automatización y creación de prototipos (Nick, s.f.).

En la práctica realizada, el ESP32 se utilizó como unidad de control para recibir las señales provenientes de dos botones pulsadores y controlar diferentes dispositivos de salida. Inicialmente, los botones permitieron controlar el encendido y apagado de un LED. Posteriormente, se utilizó un transistor IRLZ44N para controlar un motor y finalmente un módulo relé para controlar un foco de esta manera, el ESP32 permitió implementar un sistema básico de control mediante entradas y salidas digitales (Nick, s.f.).

## Programación del ESP32

PlatformIO es un entorno de desarrollo utilizado para la programación de microcontroladores y diferentes plataformas de sistemas embebidos puede integrarse como una extensión en entornos de desarrollo como Visual Studio Code, proporcionando herramientas para crear y administrar proyectos, gestionar librerías, compilar programas y cargar el código en la tarjeta de desarrollo además, permite configurar cada proyecto mediante el archivo platformio.ini, donde se especifican aspectos como la placa y las características necesarias para su funcionamiento (PlatformIO, s.f.).

En este proyecto se utilizó PlatformIO dentro de Visual Studio Code para desarrollar y cargar los programas utilizados con el ESP32 su elección permitió mantener una estructura organizada del proyecto y facilitar la administración de las librerías y configuraciones necesarias para la programación también proporcionó herramientas como la compilación del código y el monitor serial, útiles para comprobar el funcionamiento del programa durante las pruebas (PlatformIO, s.f.).

## Simulación

Wokwi es una plataforma en línea que permite diseñar, programar y simular proyectos electrónicos directamente desde un navegador web la herramienta permite trabajar con diferentes placas de desarrollo, entre ellas Arduino, ESP32, STM32 y Raspberry Pi Pico, además de integrar componentes electrónicos y conexiones para comprobar virtualmente el funcionamiento de un circuito (Group, 2023).

**Jesús Ángel Bonilla Duran** **Viridiana Santibañez Carrillo** **Jocelyn Rodriguez Torres**

En este proyecto se utilizó Wokwi para diseñar y simular los circuitos empleados con el ESP32, permitiendo realizar pruebas del programa y verificar las conexiones de los componentes antes de llevar a cabo el montaje físico de esta manera, la simulación ayudó a detectar posibles errores de programación o conexión durante el desarrollo del proyecto.

### Push buttons como entradas digitales

Un push button o botón pulsador es un interruptor eléctrico que funciona mediante una acción física de presión se utiliza en circuitos de control para iniciar o detener diferentes procesos al ejercer presión sobre el botón, su mecanismo interno modifica el estado de los contactos eléctricos, permitiendo cerrar o interrumpir el circuito (Group, 2023).

Los push buttons pueden encontrarse en configuraciones normalmente abiertas (NO) o normalmente cerradas (NC) en un contacto normalmente abierto, el circuito permanece abierto mientras el botón no es presionado y los contactos se cierran al ejercer presión en cambio, un contacto normalmente cerrado permanece conectado mientras el botón está en reposo y se abre al presionarlo (Group, 2023).

## Desarrollo

### Material requerido

- 1 tarjeta ESP32.
- 1 LED o foco.
- 1 resistencia de 1 kΩ.
- 2 botones pulsadores.
- Protoboard.
- Cables de conexión.
- Computadora con Visual Studio Code y PlatformIO.
- Simulador Wokwi.
- Arduino Framework.

*Tabla 1. Configuración de pines*

| Elemento | Pin GPIO | Configuración |
|---|---|---|
| LED | GPIO 23 | Salida digital |
| Botón 1 | GPIO 21 | Entrada digital con INPUT_PULLUP |
| Botón 2 | GPIO 19 | Entrada digital con INPUT_PULLUP |

**Jesús Ángel Bonilla Duran** **Viridiana Santibañez Carrillo** **Jocelyn Rodriguez Torres**

Como primera parte, se identificaron los pines GPIO del ESP32 que se utilizarían para conectar el LED y los dos botones pulsadores, se asignó el GPIO 23 como salida digital para controlar el LED, mientras que los GPIO 21 y 19 se configuraron como entradas digitales para los botones 1 y 2, respectivamente.

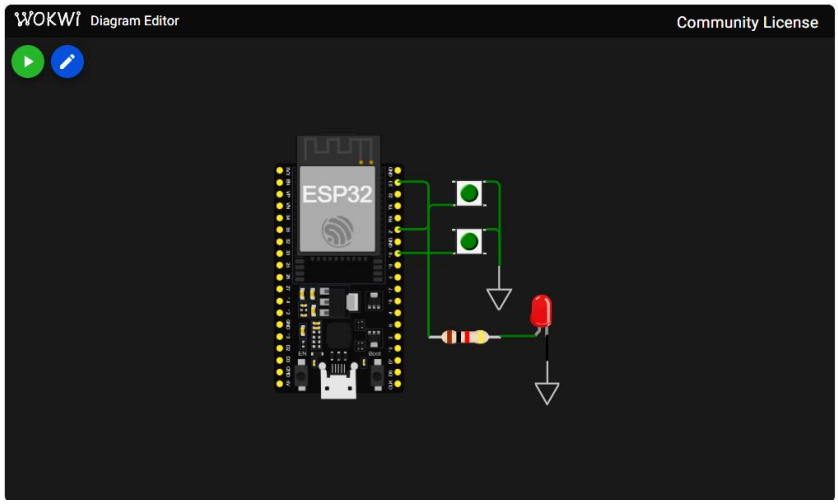



*Figura 1 Circuito de control de un LED mediante dos botones pulsadores utilizando el ESP32.*

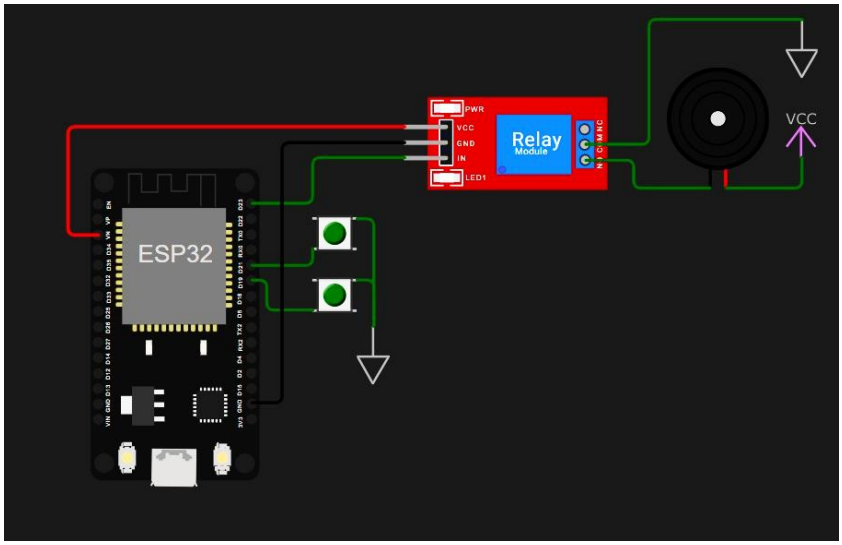



*Figura 2. Circuito simulado en Wokwi para el control de un motor mediante el ESP32 y un módulo de relevador.*

**Jesús Ángel Bonilla Duran** **Viridiana Santibañez Carrillo** **Jocelyn Rodriguez Torres**

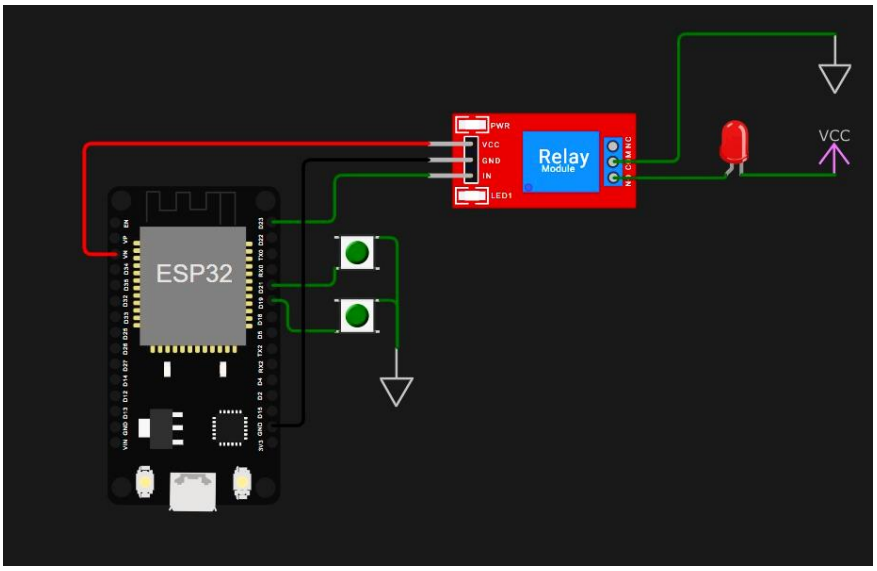



*Figura 3. Circuito de control de un LED mediante un módulo de relevador y dos botones pulsadores utilizando el ESP32.*

Enseguida, se configuraron las entradas y salidas digitales mediante las instrucciones correspondientes del framework de Arduino se utilizó la configuración INPUT_PULLUP para detectar las pulsaciones de los botones y se establecieron las condiciones para encender y apagar el LED.

A continuación, se muestra el código empleado para el desarrollo de la práctica.

**main.cpp**

```
#include <Arduino.h>

#define LED 23
#define SW1 21
#define SW2 19

#define PRENDE_LED digitalWrite(LED, HIGH)
#define APAGA_LED digitalWrite(LED, LOW)
#define PRESSED(x) digitalRead(x) == 0

void setup() {

  Serial.begin(115200);
  Serial.println("Hello, ESP32!");

  pinMode(LED, OUTPUT);
  pinMode(SW1, INPUT_PULLUP);
  pinMode(SW2, INPUT_PULLUP);
}

void loop() {

  if (PRESSED(SW1)) {
    PRENDE_LED;
    Serial.println("Boton 1 pulsado");
    delay(100);
  }

  if (PRESSED(SW2)) {
    APAGA_LED;
    Serial.println("Boton 2 pulsado");
delay(100);
  }
}
```

**Jesús Ángel Bonilla Duran** **Viridiana Santibañez Carrillo** **Jocelyn Rodriguez Torres**

### Descripción del código

*Tabla 2.Funcionamiento del código*

| Instrucción o elemento | Función |
|---|---|
| `#define LED 23` | Asigna el GPIO 23 al LED. |
| `#define SW1 21` | Asigna el GPIO 21 al botón 1. |
| `#define SW2 19` | Asigna el GPIO 19 al botón 2. |
| `PRENDE_LED` | Establece el nivel HIGH en el pin del LED. |
| `APAGA_LED` | Establece el nivel LOW en el pin del LED. |
| `PRESSED(x)` | Comprueba si la entrada indicada se encuentra en LOW. |
| `Serial.begin(115200)` | Inicia la comunicación serial a 115200 baudios. |
| `pinMode()` | Configura los pines como entradas o salidas. |
| `digitalRead()` | Lee el estado lógico de una entrada. |
| `digitalWrite()` | Establece el estado lógico de una salida. |
| `delay(100)` | Introduce una pausa de 100 milisegundos. |

## Resultados

Durante la práctica se trabajó con las entradas y salidas digitales del ESP32 para controlar un LED mediante dos botones pulsadores, se estableció que el botón 1 permite encender el LED y el botón 2 apagarlo, utilizando los pines GPIO asignados en el programa.

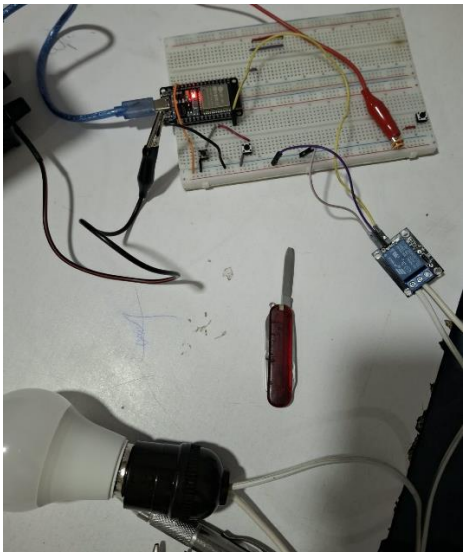


*Figura 4.Montaje físico cuando el foco está apagado*

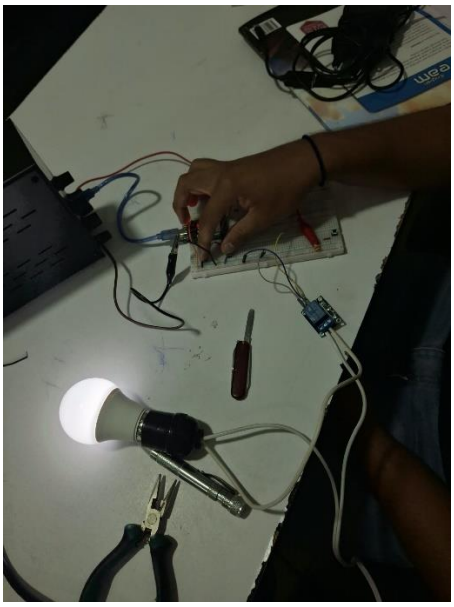


*Figura 5. Foco encendido*

También se implementó la comunicación serial para mostrar mensajes cuando se detecta la pulsación de cada botón, con esta actividad se abordó la relación entre las señales de entrada, las instrucciones del programa y la respuesta de la salida digital.

## Observaciones y Conclusiones

Durante el desarrollo de la práctica se consideró importante realizar correctamente las conexiones del LED y los botones, así como configurar los pines del ESP32 de acuerdo con su función, se utilizó INPUT_PULLUP para detectar las pulsaciones, por lo que el estado lógico de la entrada cambia a LOW cuando se presiona un botón, además, el programa incluye mensajes en el monitor serial para identificar las entradas activadas.

## Referencias

[1] (Nick, s.f.) *The ESP32 chip explained: Advantages and applications*.

[2] PlatformIO. (s.f.). PlatformIO: Professional collaborative platform for embedded development.

[3] Maxi. (s.f.). Introducción a PlatformIO con Visual Studio Code.

[4] Group, u. (2023). Wokwi: Simulador en línea de Arduino, ESP32, STM32 y Raspberry Pi Pico.